Лабораторная работа №2: Основные структуры данных и управляющие конструкции Python

Часть 1: Основы управления потоком выполнения

1.1 Условные операторы

```
# Пример: Калькулятор с проверкой деления на ноль
print("=== Простой калькулятор ===")
a = float(input("Введите первое число: "))
b = float(input("Введите второе число: "))
operation = input("Выберите операцию (+, -, *, /): ")

if operation == "+":
    result = a + b
    print(f"{a} + {b} = {result}")
elif operation == "-":
    result = a - b
    print(f"{a} - {b} = {result}")
elif operation == "*":
    result = a * b
    print(f"{a} * {b} = {result}")
elif operation == "/":
    if b != 0:  # Проверка деления на ноль
        result = a / b
        print(f"{a} / {b} = {result:.2f}")  # :.2f - округление до 2 знаков
    else:
        print("Ошибка: деление на ноль!")
else:
    print("Неизвестная операция!")

# Пример: Определение категории возраста
age = int(input("\nВведите ваш возраст: "))
```

```
if age < 0:
    print("Некорректный возраст!")
elif age <= 12:
    print("Ребенок")
elif age <= 17:
    print("Подросток")
elif age <= 64:
    print("Взрослый")
else:
    print("Пожилой человек")
```

Задание 1.1: Напишите программу, которая определяет тип треугольника по длинам его сторон.

* Равносторонний (все стороны равны)
* Равнобедренный (две стороны равны)
* Разносторонний (все стороны разные)
* Не треугольник (сумма двух сторон меньше третьей)

1.2 Циклы

```
# Пример: Анализ вводимых данных с использованием while
print("\n=== Анализ чисел ===")
positive_count = 0
negative_count = 0
zeros_count = 0

while True:
    user_input = input("Введите число (или 'стоп' для завершения): ")

    if user_input.lower() == 'стоп':
        break

    try:
```

```
        number = float(user_input)
        if number > 0:
            positive_count += 1
        elif number < 0:
            negative_count += 1
        else:
            zeros_count += 1
    except ValueError:
        print("Ошибка! Введите число или 'стоп'")

print(f"\nСтатистика введенных чисел:")
print(f"Положительных: {positive_count}")
print(f"Отрицательных: {negative_count}")
print(f"Нулей: {zeros_count}")

# Пример: Поиск простых чисел с использованием for
print("\n=== Поиск простых чисел в диапазоне ===")
start = int(input("Начало диапазона: "))
end = int(input("Конец диапазона: "))

print(f"Простые числа от {start} до {end}:")
for num in range(start, end + 1):
    if num > 1:
        is_prime = True
        for i in range(2, int(num  0.5) + 1):
            if num % i == 0:
                is_prime = False
                break
        if is_prime:
            print(num, end=" ")
print()  # Переход на новую строку
```

Задание 1.2: Напишите программу, которая генерирует таблицу умножения от 1 до N, где N вводит пользователь. Выводите таблицу в красивом формате.

Часть 2: Строки и их обработка

Пример: Анализ текста

```
print("\n=== Анализ текста ===")
text = input("Введите текст: ")

Базовый анализ
print(f"Длина текста: {len(text)} символов")
print(f"Текст в верхнем регистре: {text.upper()}")
print(f"Текст в нижнем регистре: {text.lower()}")

Подсчет символов
vowels = "аеёиоуыэюяaeiou"
consonants = "бвгджзйклмнпрстфхцчшщbcdfghjklmnpqrstvwxyz"
digits = "0123456789"

vowel_count = sum(1 for char in text.lower() if char in vowels)
consonant_count = sum(1 for char in text.lower() if char in consonants)
digit_count = sum(1 for char in text if char in digits)
space_count = text.count(" ")

print(f"\nСтатистика символов:")
print(f"Гласные буквы: {vowel_count}")
print(f"Согласные буквы: {consonant_count}")
print(f"Цифры: {digit_count}")
print(f"Пробелы: {space_count}")
```

Пример: Палиндром

```
word = input("\nВведите слово для проверки на палиндром: ")
cleaned_word = word.lower().replace(" ", "")
is_palindrome = cleaned_word == cleaned_word[::-1]

if is_palindrome:
    print(f"'{word}' - это палиндром!")
else:
    print(f"'{word}' - не палиндром.")
```

Работа со срезами

```
print("\n=== Работа со срезами строк ===")
sample_text = "Программирование на Python - это увлекательно!"
print(f"Исходная строка: {sample_text}")
print(f"Первые 10 символов: {sample_text[:10]}")
print(f"Последние 5 символов: {sample_text[-5:]}")
print(f"Каждый второй символ: {sample_text[::2]}")
print(f"Строка в обратном порядке: {sample_text[::-1]}")
```

Задание 2.1: Напишите программу, которая:

1. Принимает предложение от пользователя
2. Находит самое длинное слово в предложении
3. Подсчитывает, сколько раз каждая буква встречается в предложении (игнорируя регистр)

Часть 3: Списки и их применение

Пример: Работа со списком студентов

```
print("\n=== Управление списком студентов ===")
students = ["Иванов Иван", "Петрова Мария", "Сидоров Алексей", "Кузнецова Анна"]

 Основные операции со списком
print(f"Исходный список: {students}")
print(f"Количество студентов: {len(students)}")

 Добавление элементов
new_student = input("\nВведите ФИО нового студента: ")
students.append(new_student)
print(f"После добавления: {students}")

 Удаление элементов
remove_name = input("Введите ФИО студента для удаления: ")
if remove_name in students:
    students.remove(remove_name)
    print(f"После удаления: {students}")
else:
    print("Студент не найден!")

 Сортировка
students.sort()
print(f"\nОтсортированный список: {students}")

 Поиск
search_name = input("Введите ФИО для поиска: ")
if search_name in students:
    position = students.index(search_name)
    print(f"Студент найден на позиции {position + 1}")
else:
    print("Студент не найден")
```

Создание списка через генератор

```
print("\n=== Генерация списков ===")
```

Список квадратов чисел от 1 до 10

```
squares = [x2 for x in range(1, 11)]
print(f"Квадраты чисел: {squares}")
```

Список четных чисел от 1 до 20

```
even_numbers = [x for x in range(1, 21) if x % 2 == 0]
print(f"Четные числа: {even_numbers}")
```

Обработка списка чисел

```
numbers = [12, 45, 23, 67, 34, 89, 56]
print(f"\nИсходный список чисел: {numbers}")
print(f"Сумма: {sum(numbers)}")
print(f"Максимум: {max(numbers)}")
print(f"Минимум: {min(numbers)}")
print(f"Среднее значение: {sum(numbers)/len(numbers):.2f}")
```

Сортировка по разным критериям

```
print(f"Сортировка по возрастанию: {sorted(numbers)}")
print(f"Сортировка по убыванию: {sorted(numbers, reverse=True)}")
```

Задание 3.1: Создайте программу для учета оценок студентов:

1. Храните в словаре, где ключ - имя студента, значение - список его оценок
2. Реализуйте функции: добавить студента, добавить оценку, удалить студента, вывести средний балл студента, вывести всех студентов с их средними баллами
3. Определите студента с самым высоким средним баллом

Часть 4: Словари и множества

Пример: Телефонный справочник

```
print("\n=== Телефонный справочник ===")
phonebook = {
    "Иванов Иван": "+7-911-123-45-67",
    "Петрова Мария": "+7-912-987-65-43",
    "Сидоров Алексей": "+7-913-456-78-90"
}
```

Основные операции

```
print("Текущие контакты:")
for name, phone in phonebook.items():
    print(f"{name}: {phone}")
```

Добавление/изменение контакта

```
new_name = input("\nВведите имя нового контакта: ")
new_phone = input("Введите телефон: ")
phonebook[new_name] = new_phone
print(f"Контакт '{new_name}' добавлен/обновлен!")
```

Поиск контакта

```
search_name = input("\nВведите имя для поиска: ")
if search_name in phonebook:
    print(f"Найден контакт: {search_name} - {phonebook[search_name]}")
else:
    print("Контакт не найден")
```

Удаление контакта

```
remove_name = input("\nВведите имя для удаления: ")
if remove_name in phonebook:
    del phonebook[remove_name]
    print(f"Контакт '{remove_name}' удален!")
```

```
else:
    print("Контакт не найден")

print(f"\nИтоговый список контактов ({len(phonebook)}):")
for name in sorted(phonebook.keys()):
    print(f"{name}: {phonebook[name]}")
```

Пример: Множества для работы с уникальными данными

```
print("\n=== Работа с множествами ===")
```

Списки студентов разных групп

```
group1 = {"Иванов", "Петров", "Сидоров", "Кузнецов"}
group2 = {"Петров", "Смирнов", "Кузнецов", "Васильев"}

print(f"Группа 1: {group1}")
print(f"Группа 2: {group2}")
```

Операции с множествами

```
print(f"\nСтуденты, которые учатся в обеих группах (пересечение): {group1 & group2}")
print(f"Все уникальные студенты (объединение): {group1 | group2}")
print(f"Студенты только в группе 1 (разность): {group1 - group2}")
print(f"Студенты только в группе 2 (разность): {group2 - group1}")
print(f"Студенты, которые учатся только в одной группе (симметрическая разность): {group1 ^ group2}")
```

Пример: Удаление дубликатов из списка

```
duplicates_list = [1, 2, 2, 3, 4, 4, 4, 5, 6, 6]
unique_set = set(duplicates_list)
unique_list = list(unique_set)
print(f"\nИсходный список с дубликатами: {duplicates_list}")
print(f"Уникальные элементы: {sorted(unique_list)}")
```

Задание 4.1: Создайте систему учета товаров на складе:

1. Используйте словарь, где ключ - название товара, значение - словарь с полями: количество, цена, категория

2. Реализуйте функции: добавить товар, изменить количество, удалить товар, посчитать общую стоимость всех товаров, найти самый дорогой товар

3. Используйте множества для хранения уникальных категорий товаров

Итоговый проект: "Система управления библиотекой"

Система управления библиотекой

Требования:

1. Хранить информацию о книгах: название, автор, год, статус (доступна/выдана)

2. Хранить информацию о читателях: ФИО, номер билета, список взятых книг

3. Реализовать функции:

- Добавление/удаление книги
- Регистрация/удаление читателя
- Выдача книги читателю
- Возврат книги в библиотеку
- Поиск книг по названию/автору
- Показать все книги с указанием статуса
- Показать всех читателей с их книгами

```
import json

from datetime import datetime, timedelta


class LibrarySystem:

    def __init__(self):

        self.books = {}  # {id: {title, author, year, status, due_date}}

        self.readers = {}  # {reader_id: {name, ticket, borrowed_books}}

        self.next_book_id = 1

        self.next_reader_id = 1


    def add_book(self, title, author, year):
```

```
        book_id = self.next_book_id
        self.books[book_id] = {
            'title': title,
            'author': author,
            'year': year,
            'status': 'available',
            'due_date': None
        }
        self.next_book_id += 1
        print(f"Книга '{title}' добавлена с ID {book_id}")

    def register_reader(self, name):
        reader_id = self.next_reader_id
        self.readers[reader_id] = {
            'name': name,
            'ticket': f"T{reader_id:03d}",
            'borrowed_books': []
        }
        self.next_reader_id += 1
        print(f"Читатель '{name}' зарегистрирован с ID {reader_id}")

    def borrow_book(self, reader_id, book_id):
        if reader_id not in self.readers:
            print("Читатель не найден!")
            return

        if book_id not in self.books:
            print("Книга не найдена!")
            return

        if self.books[book_id]['status'] != 'available':
            print("Книга уже выдана!")
```

```
        return

    Выдаем книгу на 14 дней
    due_date = datetime.now() + timedelta(days=14)
    self.books[book_id]['status'] = 'borrowed'
    self.books[book_id]['due_date'] = due_date.strftime("%Y-%m-%d")
    self.readers[reader_id]['borrowed_books'].append(book_id)

    print(f"Книга '{self.books[book_id]['title']}' выдана читателю "
          f"{self.readers[reader_id]['name']} до {due_date.strftime('%d.%m.%Y')}")

def return_book(self, reader_id, book_id):
    if reader_id not in self.readers:
        print("Читатель не найден!")
        return

    if book_id not in self.books:
        print("Книга не найдена!")
        return

    if book_id not in self.readers[reader_id]['borrowed_books']:
        print("Эта книга не была выдана данному читателю!")
        return

    self.books[book_id]['status'] = 'available'
    self.books[book_id]['due_date'] = None
    self.readers[reader_id]['borrowed_books'].remove(book_id)

    print(f"Книга '{self.books[book_id]['title']}' возвращена")

def search_books(self, keyword):
    print(f"\nРезультаты поиска по '{keyword}':")
```

```
        found = False
        for book_id, book in self.books.items():
            if (keyword.lower() in book['title'].lower() or
                keyword.lower() in book['author'].lower()):
                status = "✓ доступна" if book['status'] == 'available' else f"✗ выдана до {book['due_date']}"
                print(f"ID: {book_id}, '{book['title']}', {book['author']}, {book['year']} - {status}")
                found = True

        if not found:
            print("Книги не найдены")

    def show_all_books(self):
        print("\n=== Все книги в библиотеке ===")
        for book_id, book in self.books.items():
            status = "✓ доступна" if book['status'] == 'available' else f"✗ выдана до {book['due_date']}"
            print(f"ID: {book_id}, '{book['title']}', {book['author']}, {book['year']} - {status}")

    def show_all_readers(self):
        print("\n=== Все читатели ===")
        for reader_id, reader in self.readers.items():
            print(f"\nID: {reader_id}, {reader['name']}, Билет: {reader['ticket']}")
            if reader['borrowed_books']:
                print("Взятые книги:")
                for book_id in reader['borrowed_books']:
                    print(f"  - {self.books[book_id]['title']}")
            else:
                print("Нет взятых книг")
```

Пример использования системы

```
def main():
    library = LibrarySystem()
```

```
Добавляем несколько книг
    library.add_book("Мастер и Маргарита", "Михаил Булгаков", 1967)
    library.add_book("Преступление и наказание", "Федор Достоевский", 1866)
    library.add_book("1984", "Джордж Оруэлл", 1949)

    Регистрируем читателей
    library.register_reader("Иванов Иван")
    library.register_reader("Петрова Мария")

    Показываем все книги
    library.show_all_books()

    Выдаем книгу
    library.borrow_book(1, 1)

    Поиск книг
    library.search_books("маргарита")

    Показываем всех читателей
    library.show_all_readers()

    Возвращаем книгу
    library.return_book(1, 1)

    Показываем книги после возврата
    library.show_all_books()

if __name__ == "__main__":
    main()
```

Задания для самостоятельной работы:

1. Задание А: Улучшите систему управления библиотекой:
   * Добавьте проверку на просроченные книги
   * Реализуйте систему штрафов за просрочку
   * Добавьте возможность сохранения данных в файл и загрузки из файла

2. Задание Б: Создайте программу для анализа текстового файла:
   * Подсчитайте количество слов, предложений, абзацев
   * Найдите топ-10 самых частых слов
   * Определите самое длинное и самое короткое предложение
   * Сохраните результаты анализа в отдельный файл

3. Задание В: Разработайте систему опросов:
   * Храните вопросы и варианты ответов в словаре
   * Поддерживайте разные типы вопросов (один ответ, несколько ответов, текстовый ответ)
   * Сохраняйте результаты ответов
   * Генерируйте статистику по ответам